

Secure Exit

SIMATS
ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

SANGEPU VENKATA PRAVEEN
192373013RD

CO3-AT1-Debugging and Optimization

[← Back](#)

QUESTIONS

Q1

ATM Withdrawal – Exception
Handling Debugging
10 marks

Q2

Hospital Registration –
NullPointerException
Debugging
10 marks

Q3

E-Commerce Product Search –
Array Exception Debugging
10 marks

Q4

Railway Reservation –
Synchronization Debugging
10 marks

Q5

Bank Account – Race Condition
Debugging
10 marks

5/5 answered

Q5

Bank Account – Race Condition
Debugging10
marks

A bank account contains ₹10,000. Two ATM threads simultaneously attempt to withdraw ₹7,000 and ₹6,000.

The following program contains a race condition.

Task:

1. Identify the race condition.
2. Identify the critical section.
3. Explain why both threads may pass the balance check.
4. Modify the complete program using synchronized.
5. Ensure that the balance can never become negative.
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class BankAccount {  
  
    int balance = 10000;  
  
    void withdraw(int amount) {  
  
        if (balance >= amount) {  
  
            System.out.println(  
                Thread.currentThread().getName()  
                + " withdrawing " + amount);  
        }  
    }  
}
```

```
try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Interrupted");
}

balance = balance - amount;

System.out.println(
    Thread.currentThread().getName()
    + " withdrawal successful");
}
else {
    System.out.println(
        Thread.currentThread().getName()
        + " insufficient balance");
    }
}
}

class ATM extends Thread {

    BankAccount account;
    int amount;

    ATM(BankAccount account, int amount) {
        this.account = account;
        this.amount = amount;
    }

    public void run() {
        account.withdraw(amount);
    }
}

class BankApplication {

    public static void main(String[] args)
        throws InterruptedException {

        BankAccount account = new BankAccount();

        ATM atm1 = new ATM(account, 7000);
        ATM atm2 = new ATM(account, 6000);
```

```

atm1.setName("ATM-1");
atm2.setName("ATM-2");

atm1.start();
atm2.start();

atm1.join();
atm2.join();

System.out.println(
"Final Balance = " + account.balance);
}
}

```

Correct the program so that only one withdrawal succeeds.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```

1 class BankAccount{
2     int balance = 10000;
3     synchronized void withdraw(int amount)
4     {
5         if (balance >= amount){
6             System.out.println(Thread.currentThread().getName() + " is withdrawing " + amount);
7             try{
8                 Thread.sleep(100);
9             } catch (InterruptedException e) {
10                System.out.println("InterruptedException");
11            }
12            balance = balance - amount;
13            System.out.println(Thread.currentThread().getName() + " has withdrawn " + amount);
14        } else {
15            System.out.println(Thread.currentThread().getName() + " cannot withdraw " + amount + " because balance is " + balance);
16        }
17    }
18    public static void main(String[] args)
19    {
20        BankAccount account = new BankAccount();
21        ATM atm1 = new ATM(account, 7000);
22        ATM atm2 = new ATM(account, 6000);
23        atm1.setName("ATM-1");
24        atm2.setName("ATM-2");
25        atm1.start();
26        atm2.start();
27    }
28 }

```

```
25         atm1.join();
26         atm2.join();
27         System.out.println("Final Balance
28     }
29 }
30 class ATM extends Thread {
31     BankAccount account;
32     int amount;
33     ATM(BankAccount account, int amount) {
34         this.account = account;
35         this.amount = amount;
36     }
37     public void run() {
38         account.withdraw(amount);
39     }
40 }
```



Test Input

stdin input (optional)



Output

```
ATM-1 withdrawing 7000
ATM-1 withdrawal successful
ATM-2 insufficient balance
Final Balance = 3000
```

 Pending manual review

 Submitted

Secure Exit



SIMATS Online Programming Lab

JAVA PROGRAMMING



SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS

Q1

ATM Withdrawal – Exception
Handling Debugging
10 marks

**Q2**

Hospital Registration –
NullPointerException
Debugging
10 marks

**Q3**

E-Commerce Product Search –
Array Exception Debugging
10 marks

**Q4**

Railway Reservation –
Synchronization Debugging
10 marks

**Q5**

Bank Account – Race Condition
Debugging
10 marks



5/5 answered

Q4

Railway Reservation – Synchronization Debugging

10
marks

A railway reservation system has only one available seat. Two customers attempt to reserve the seat simultaneously.

The following complete Java program contains a synchronization problem.

Task:

1. Identify the race condition.
2. Debug the program using the synchronized keyword.
3. Ensure that only ONE customer can reserve the available seat.
4. The second customer must receive "No seat available".
5. Submit the COMPLETE corrected Java program.

Faulty Program:

```
class Reservation {  
  
    int seats = 1;  
  
    void bookSeat(String customer) {  
  
        if (seats > 0) {  
  
            System.out.println(customer  
                + " is booking a seat");  

```

```
try {
    Thread.sleep(100);
}
catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}

seats--;

System.out.println(customer
    + " booking successful");

} else {
    System.out.println(customer
        + " - No seat available");
}
}
}

class Customer extends Thread {

    Reservation reservation;
    String customerName;

    Customer(Reservation reservation, String
        customerName) {
        this.reservation = reservation;
        this.customerName = customerName;
    }

    public void run() {
        reservation.bookSeat(customerName);
    }
}

class RailwayReservation {

    public static void main(String[] args) {

        Reservation reservation = new Reservation();

        Customer c1 =
            new Customer(reservation, "Customer 1");

        Customer c2 =
            new Customer(reservation, "Customer 2");
```

```
c1.start();
c2.start();
}
}
```

The corrected program must protect the shared seat resource using synchronization.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 class Reservation{
2     int seats = 1;
3     synchronized void bookSeat(String customerName) {
4         if (seats > 0) {
5             System.out.println(customerName + " booked a seat.");
6             try {
7                 Thread.sleep(100);
8             } catch (InterruptedException e) {
9                 System.out.println("Thread interrupted");
10            }
11            seats--;
12            System.out.println(customerName + " left the seat.");
13        } else {
14            System.out.println(customerName + " cannot book a seat as all seats are occupied.");
15        }
16    }
17    public static void main(String[] args) {
18        Reservation reservation = new Reservation();
19        Customer c1 = new Customer(reservation, "Customer 1");
20        Customer c2 = new Customer(reservation, "Customer 2");
21        c1.start();
22        c2.start();
23    }
24 }
25 class Customer extends Thread{
26     Reservation reservation;
27     String customerName;
28     Customer(Reservation reservation, String customerName) {
29         this.reservation = reservation;
30         this.customerName = customerName;
31     }
32     public void run() {
```

```
33         reservation.bookSeat(customerName)
34
35     }
36 }
```

Test Input

stdin input (optional)

Output

```
Customer 1 is booking a seat
Customer 1booking successful
Customer 2 - No seat available
```

 Pending manual review

 Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.

Secure Exit



SIMATS Online Programming Lab

JAVA PROGRAMMING



SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS

Q1

ATM Withdrawal – Exception
Handling Debugging
10 marks

**Q2**

Hospital Registration –
NullPointerException
Debugging
10 marks

**Q3**

E-Commerce Product Search –
Array Exception Debugging
10 marks

**Q4**

Railway Reservation –
Synchronization Debugging
10 marks

**Q5**

Bank Account – Race Condition
Debugging
10 marks



5/5 answered

Q3

E-Commerce Product Search – Array Exception Debugging

10
marks

An online shopping application allows customers to select a product using a product number from 1 to 5.

The following program contains an array indexing error.

Task:

1. Identify the error.
2. Correct the product-number and array-index logic.
3. Handle invalid product numbers.
4. Handle non-numeric input.
5. Submit the COMPLETE corrected Java program.
6. The program must display "Search completed" after processing.

Faulty Program:

```
import java.util.Scanner;

class ProductSearch {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String[] products = {
            "Laptop",
            "Mobile",
            "Tablet",
            "Headphone",
```

```
"Keyboard"
```

```
};
```

```
System.out.print("Enter product number (1-5): ");
```

```
int productNumber = sc.nextInt();
```

```
System.out.println("Selected product: "
```

```
+ products[productNumber]);
```

```
System.out.println("Search completed");
```

```
}
```

```
}
```

Correct the program so that product numbers 1, 2, 3, 4 and 5 correctly access the corresponding products.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 class ProductSearch{
3     public static void main(String[] args)
4     {
5         Scanner sc = new Scanner(System.in);
6         String[] products = {"Laptop", "Mobile", "TV", "Refrigerator", "Washing Machine"};
7
8         try{
9             int productNumber = sc.nextInt();
10            if (productNumber >= 1 && productNumber <= products.length)
11                System.out.println("Selected product: " + products[productNumber]);
12            } else {
13                System.out.println("Invalid product number");
14            }
15        } catch (java.util.InputMismatchException e) {
16            System.out.println("Invalid input");
17        } catch (ArrayIndexOutOfBoundsException e) {
18            System.out.println("Invalid product number");
19        } finally {
20            System.out.println("Search completed");
21        }
22 }
```

Test Input

1



Output

```
Selected product: Laptop  
Search completed
```

 Pending manual review

 Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.

Secure Exit



SIMATS Online Programming Lab

JAVA PROGRAMMING



SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS

Q1

ATM Withdrawal – Exception
Handling Debugging
10 marks

**Q2**

Hospital Registration –
NullPointerException
Debugging
10 marks

**Q3**

E-Commerce Product Search –
Array Exception Debugging
10 marks

**Q4**

Railway Reservation –
Synchronization Debugging
10 marks

**Q5**

Bank Account – Race Condition
Debugging
10 marks



5/5 answered

Q2

Hospital Registration – NullPointerException Debugging

10
marks

A hospital patient registration application crashes
when patient information is unavailable.

Debug the following complete Java program.

Task:

1. Identify the runtime error.
2. Modify the program to handle a missing patient name.
3. Handle invalid age input.
4. Use try-catch-finally appropriately.
5. The program must always display "Registration completed".
6. Submit the COMPLETE corrected Java program.

Faulty Program:

```
import java.util.Scanner;

class PatientRegistration {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter patient name: ");
        String inputName = sc.nextLine();

        String name;
```

```
if (inputName.equals("NULL")) {  
    name = null;  
} else {  
    name = inputName;  
}  
  
System.out.print("Enter age: ");  
int age = sc.nextInt();  
  
System.out.println("Patient Name: "  
    + name.toUpperCase());  
  
if (age >= 18) {  
    System.out.println("Adult patient");  
} else {  
    System.out.println("Minor patient");  
}  
  
System.out.println("Registration completed");  
}  
}
```

The corrected program must prevent
NullPointerException and handle invalid age input
without terminating unexpectedly.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;  
2 class PatientRegistration{  
3     public static void main(String[] args)  
4     {  
5         Scanner sc = new Scanner(System.in);  
6         try{  
7             String inputName = sc.nextLine();  
8             String name;  
9             if (inputName.equals("NULL")  
10                name = "Unknown";  
11            } else {  
12                name = inputName;  
13            }  
14        }  
15    }  
16 }
```

```
15         int age = sc.nextInt();
16         System.out.println("Patient Name: " + name);
17         if (age >= 18){
18             System.out.println("Adult patient");
19         } else {
20             System.out.println("Minor patient");
21         }
22     } catch (java.util.InputMismatchException e) {
23         System.out.println("Invalid input");
24     } catch (NullPointerException e) {
25         System.out.println("Patient name is null");
26     } catch (Exception e) {
27         System.out.println("Unexpected error");
28     } finally {
29         System.out.println("Registration completed");
30     }
31 }
32 }
```



Test Input

Ravi
25

Output

Patient Name: RAVI
Adult patient
Registration completed

 Pending manual review

 Submitted

Secure Exit



SIMATS Online Programming Lab

JAVA PROGRAMMING



SANGEPU VENKATA PRAVEEN
192373013RD

QUESTIONS

Q1

ATM Withdrawal – Exception Handling Debugging
10 marks

**Q2**

Hospital Registration – NullPointerException Debugging
10 marks

**Q3**

E-Commerce Product Search – Array Exception Debugging
10 marks

**Q4**

Railway Reservation – Synchronization Debugging
10 marks

**Q5**

Bank Account – Race Condition Debugging
10 marks



5/5 answered

Q1

ATM Withdrawal – Exception Handling Debugging

10
marks

An ATM application is used to process cash withdrawals. The following Java program contains programming errors.

Task:

1. Run the given program and identify the errors.
2. Debug and modify the program.
3. Submit the COMPLETE corrected Java program.
4. The corrected program must handle insufficient balance, zero withdrawal, invalid input, and unexpected errors.
5. Use appropriate try-catch-finally exception handling.
6. The corrected program must execute successfully for all valid test cases.

Faulty Program:

```
import java.util.Scanner;

class ATM {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter account balance: ");
        int balance = sc.nextInt();
```

```
System.out.print("Enter withdrawal amount: ");
int amount = sc.nextInt();

try {
    int remaining = balance / amount;

    if (amount > balance) {
        System.out.println("Insufficient balance");
    } else {
        System.out.println("Withdrawal successful");
        System.out.println("Remaining balance = "
            + (balance - amount));
    }
}
catch (ArithmeticException e) {
    System.out.println("Transaction failed");
}

System.out.println("Transaction completed");
}
```

Expected correction:

The program must correctly validate the withdrawal amount before performing the transaction, handle division-by-zero and invalid input, and always display "Transaction completed" after exception handling.

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 class ATM{
3     public static void main(String[] args)
4     {
5         Scanner sc = new Scanner(System.in);
6
7         try{
8             int balance = sc.nextInt();
9
10            int amount = sc.nextInt();
11            try{
12                if (amount <= 0) {
```



```
12         System.out.println("Ir
13     }
14     else if (amount > balance)
15         System.out.println("Ir
16     }
17     else{
18         System.out.println("Re
19     }
20 }
21 catch (ArithmeticException e) {
22     System.out.println("Transa
23 }
24 }
25 catch (java.util.InputMismatchExce
26     System.out.println("Invalid in
27 }
28 catch (Exception e) {
29     System.out.println("unexpected
30 }
31 System.out.println("Transaction co
32 }
33 }
```



Test Input

10000
3000

Output

Remaining balance = 7000
Transaction completed

 Pending manual review

 Submitted

© 2026 **SIMATS Online Lab**. All rights reserved.